Assignment 2 Logbook: A Luanti Diamond Agent

Oct 20, 2026 · @Tanush Panuganti

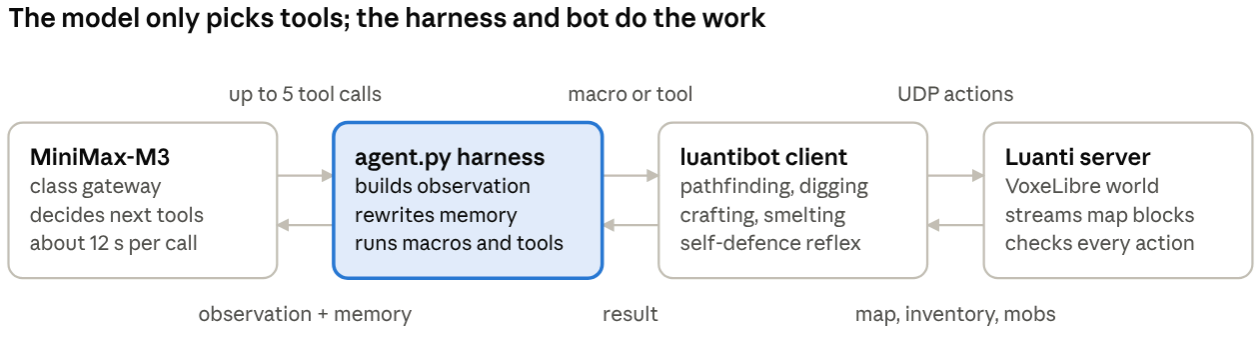
Summary

My agent took a diamond from a fresh VoxeLibre world in **3 min 29 s on the official clock, using one LLM call and about 2,650 tokens, rank 23 on the class speedrun leaderboard**. The brain is MiniMax-M3 on the class gateway. It never sees pixels: each turn it gets a short text observation and calls tools in a Python harness, which drives a headless network client that speaks the Luanti protocol directly.

Getting there took 10 runs and 28 LLM calls in total. The main lesson: the agent got fast and reliable only after I moved the known recipe for a diamond out of the model and into code. In the final runs the model's job was to call four macros in order and recover if one failed.

I built the harness with Claude Code (Claude Opus 5.5 in the desktop app). I made the design calls: what the agent sees, how coarse the tools are, what it remembers. Claude Code wrote and debugged the code and watched the runs.

Architecture



agent architecture · 4 parts, one loop per turn

Each turn runs left to right and back: the harness sends the model an observation, the model returns tool calls, and the harness runs them through the client, which talks to the server over Luanti's UDP protocol. No game window is involved.

The client is the course's starter library, a headless Python client in the style of Mineflayer. The server trusts each client to report its own position and only sanity-checks it, which is how a program without a game window can move. I added `agent.py` (the loop,

observation, memory and macros), fixed three bugs in the library, and gave it a hook for the reflex.

What it can do: tool granularity

The tools ended up in three layers. The model could call any of them, and in the end it only needed the top one.

Layer	Tools	What one call does
Macros	`make_wood_pickaxe`, `make_stone_pickaxe`, `make_iron_pickaxe`, `get_diamond`	A whole stage of the tech tree: chop ~6 logs and craft through to a pickaxe; dig down for 11 cobble and craft a pickaxe and furnace; find, mine and smelt 3 iron; tunnel to the nearest visible diamond.
Mid-level	`gather`, `craft`, `smelt`, `mine_down`, `tunnel`, `dig_to`	One job: dig N blocks of a type, craft one item (placing a crafting table if needed), staircase to a depth, tunnel straight to a coordinate.
Fine	`goto`, `dig`, `find`, `defend`, `chat`, `remember`, `forget`	One action or lookup.

I started with only the mid and fine layers, so the model planned the tech tree itself. It planned well: on its own it worked out that it needed 9 planks for sticks, a table and a pickaxe. But each step cost about 12 s of model latency, and every gap in a tool turned into a wasted turn.

The macros encode the path that never changes (wood, stone, iron, diamond). The model still decides the order, skips stages it has already done, and falls back to the finer tools when a macro fails. Two more choices kept calls down:

- **Batched calls.** The model may return up to 5 tool calls per reply, run in order; the first failure cancels the rest. In the best run it put all four macros in one reply, so the whole game took a single LLM call.
- **A reflex outside the model.** While the model thinks, the bot thread fights any mob within 5 nodes by itself. This answers the brief's "what if you are attacked mid-goto?" without spending a turn.

What it can see: the observation

Each turn the harness rebuilds a text observation of about 15 lines from the client's world model. This is the real one from the end of my first diamond run:

```
Position: (94, -52, -89)   HP: 20/20
Standing on: mcl_core:granite
Map known down to y=-96
Inventory: cobble x127, granite x14, diorite x7, ... pick_iron x1, diamond x1
Best pickaxe: pick_iron
Visible ore (nearest first):
  diamond: (95,-53,-88) 2m; (94,-54,-88) 2m; (98,-53,-78) 12m
  iron: (91,-50,-87) 4m; ...
  coal: (93,-51,-92) 3m; ...
  nearest tree log: 63m away
Craft grid: 3x3 (table open)
```

- **The wallhack is the most useful line.** The server streams map blocks around the player, solid rock included, so the client knows where ore is before it is uncovered. I show the 3 nearest of each ore with coordinates and distance. That turns "explore for a diamond" into "tunnel to (95,-53,-88)".
- **Pre-digested facts, not raw state.** Item names are shortened, the best pickaxe is named, and nearby crafting tables and furnaces are listed. The model doesn't have to work these out from a list of slots.
- **Events between turns.** After the first death I added "took N damage since last turn" and "you died at (x,y,z), your items dropped there". Before that, the model could only guess at a death from an empty inventory.
- **Left out:** screenshots, raw block grids, and any entity that is not a hostile mob. Only hostiles within 16 nodes are listed. A block grid would have cost thousands of tokens and the ore list already answers the question it would be used for.

What it remembers: no transcript at all

I never let the transcript grow, so the context window never fills. Every request is just two messages: the system prompt, and one user message holding a memory block plus the fresh observation. The model's reply is executed and then thrown away.

The memory block is rewritten in place each turn and has two parts:

- **Recent actions:** a one-line log of the last 12 tool calls with their results cut to 260 characters, e.g. `#4 craft(item=mcl_core:wood) -> {"error": "need 1 x mcl_core:tree..."}`. Older lines fall off the end.

- **Notes:** up to 15 facts the model saves itself with `remember` and deletes with `forget`, kept for the whole run.

The world state (position, inventory, visible ore) is not memory at all: it is read fresh every turn, so it can never go stale.

The payoff is a flat cost per call. Prompts stayed between about 1,800 and 3,200 tokens from the first turn to the last. In practice the model never used `remember` in a run that reached the diamond: with macros there was nothing worth carrying between at most 4 turns.

What broke

Nearly every failure was in the harness, not the model. MiniMax read errors well and adjusted, but it couldn't fix a tool that was wrong. In the order I hit them:

When	What happened	Fix
Setup	The starter code couldn't run on Windows: it loads `libzstd.so.1`, a Linux library name.	Also look for `libzstd`, which Git for Windows ships as a DLL.
Setup	Every request to the gateway failed with Cloudflare error 1010.	Cloudflare blocks Python's default User-Agent; I send my own.
Setup	Login failed with a bare "server sent DISCO".	The real reason, "server is full", arrived first and was swallowed. Reported it properly. The server has room for only the owner and the watcher, and I was still logged in.
Run 2	Dug 4 logs, got 2; then `craft planks` failed with "need mcl_core:tree".	The recipe book only knew oak, but spawn was a spruce forest. Recipes now swap in whatever species of log or plank is held.
Run 3	Lost 8 HP while crafting, then died a second later, with no mob listed within 16 nodes. Lost everything it carried.	The bot is helpless during the ~12 s the model thinks. Added the self-defence reflex and damage/death lines in the observation.

When	What happened	Fix
Run 4	Dug 8 stone, got 2 cobble.	Drops land in the hole, where the bot can't stand, and VoxeLibre only hands items to a player within about 2 nodes. Now the bot walks next to the drop and punches the item entity to pick it up: 8 dug, 7 collected.
Run 5	`defend` chased a "hostile" it could never kill.	VoxeLibre's `spider_eyes` is a cosmetic entity attached to spiders, and it matched "spider". Excluded `*_eyes`.
Run 5	Crafting a stone pickaxe failed: the crafting table was 8 m away and unreachable.	Stations now fall back to placing or crafting a new table.
Every restart	Killing the agent left a ghost player for about a minute, and the server stayed "full".	A `runs/STOP` file makes the agent log out cleanly; login retries every 5 s.
Run 8 (recorded)	Started from a PowerShell terminal, the login hung for 90 s per attempt while the official clock ran. The run itself took 4.9 min, but it dropped me to rank 33.	Launching the same script from Bash connected at once. I never found the cause.
Run 9	The bot had the diamond at 3.9 min, but the portal didn't count it until I logged the bot back in and moved it: official time 7 min 19 s.	After the diamond, the agent now moves it to another slot and stays online for 15 s. Run 10 registered at once.

The model's own mistakes were small. It asked to gather "cobble" (you mine stone to get cobble) and opened two runs with `defend` against mobs it couldn't beat with bare hands. Each cost one turn.

Cost

Across all 10 runs the agent made **28 LLM calls and used about 70,000 input and 5,400 output tokens**. That cost nothing on the class gateway. Each call took 10 to 15 s, which mattered far more than tokens.

Run	Tools available	LLM calls	Tokens in / out	Agent time	Outcome
10	macros, fresh world	1	2,404 / 249	3.8 min	Diamond; official 3 min 29 s, rank 23
9	macros, fresh world	4	10,087 / 281	3.9 min	Diamond, but the portal missed it until a re-login (7 min 19 s)
8	macros, fresh world, screen-recorded	4	12,024 / 227	4.9 min	Diamond; the login delay dropped me to rank 33
7	macros, fresh world	3	8,053 / 603	4.5 min	Diamond from an empty inventory; official 4 min 30 s, rank 25
6	macros, from a wooden pickaxe	3	8,724 / 196	3.2 min	Diamond
5	fine + mid, batched calls	2	4,980 / 1,051	3.8 min	Stopped: crafting table out of reach
4	fine + mid, batched calls	1	2,360 / 484	2.8 min	Stopped to fix drop pickup
3	fine + mid, one call per turn	7	15,854 / 1,838	4.4 min	Died once; stopped at sticks
2	fine + mid, one call per turn	2	4,053 / 195	1 min	Stopped: oak-only recipes
1	fine + mid, one call per turn	1	1,799 / 239	under 1 min	Stopped so I could restart at daytime

In the best run the model thought for 15 s and the bot acted for 200 s, then stayed online 15 s for the portal. Compare run 3: 7 calls to get from 2 logs to 4 sticks, with a death in the middle.

The official clock roughly matched the agent's own timing whenever the bot got in straight away: 4 min 30 s for run 7 and 3 min 29 s for run 10. The leaderboard keeps your latest run, not your best, so runs 8 and 9 briefly pushed me down.

Reflections

Most of the success came from the harness, not the LLM. In the best run the model's whole contribution was one reply listing the four macros, and a script could have done the same. That is the trade-off from the lecture made concrete: coarse tools took the reasoning off the model and moved the edge cases (unreachable tables, buried drops, lava next to the tunnel) into my code.

The model was still useful, in two places:

- **Early exploration.** With fine tools it planned a correct plank budget, noticed its own death, and recovered without being told. That told me what the macros needed to do.

- **Recovery.** When a macro fails, the model can drop to `dig_to`, `gather` or `craft` and then return to the macros. None of the final runs needed this, so it's untested.

Using an AI coding agent to build the AI agent worked well for code and badly for judgement. Claude Code found each bug fast once the logs showed it. But I only found out what the bot needed (drops collected, a reflex, macros) by watching it fail in the real world.

What I'd try next:

- Skip the LLM for the opening, since it never changes, saving the 15 s of thinking.
- Measure how fast the server lets the bot walk and fall. I set 5.5 nodes/s but never found the anti-cheat limit.
- Run the same harness with a weaker model to test whether the LLM matters at all once the macros exist.

Proof

The agent's own log ends with the inventory read straight from the server:

```
*** DIAMOND OBTAINED at step 1 after 3.6 min ***
Inventory: cobble x114, andesite x12, wood x11, dirt x4, stick x2,
  pick_wood x1, pick_iron x1, pick_stone x1, furnace x1, diamond x1
```

tpan459's inventory in the game client after run 10. The diamond is the blue gem in the top row, third slot; the hotbar holds the wooden, stone and iron pickaxes:



The class leaderboard recorded run 10 at 3 min 29 s, rank 23:

22	Bryan Quach	3m 29s	2026-10-14 07:49
23	Tanush Panuganti you	3m 29s	2026-10-20 03:58
24	Rohit Balaji	3m 34s	2026-10-13 04:59